

LiDAR-Based SLAM with Scan-to-Submap ICP and Pose Graph Optimization

Edward Wang

ECE 276A: Sensing & Estimation in Robotics

University of California, San Diego

Winter 2025

I. INTRODUCTION

Simultaneous Localization and Mapping (SLAM) is a fundamental problem in mobile robotics: a robot must estimate its own trajectory while simultaneously building a map of the environment using onboard sensors. Accurate SLAM is critical for autonomous navigation, exploration, and interaction with the physical world.

In this project, we implement a full 2-D LiDAR SLAM pipeline for a differential-drive robot equipped with wheel encoders, an IMU, a Hokuyo 2-D LiDAR scanner, and a Kinect RGBD camera. Our approach proceeds in four stages: (1) encoder-and-IMU odometry prediction using a differential-drive motion model, (2) scan-to-submap point-to-line ICP for refined pose estimation, (3) occupancy grid mapping and RGBD-based texture mapping, and (4) proximity-based loop closure detection followed by pose graph optimization (PGO) to reduce accumulated drift. We evaluate the pipeline on two indoor datasets and analyze the impact of each stage on trajectory and map quality.

II. PROBLEM FORMULATION

Let $T_t^{wb} \in SE(2)$ denote the pose of the robot body frame $\{b\}$ in the world frame $\{w\}$ at discrete time t . Each pose is a 3×3 homogeneous matrix encoding 2-D position (x_t, y_t) and heading θ_t :

$$T_t^{wb} = \begin{bmatrix} \cos \theta_t & -\sin \theta_t & x_t \\ \sin \theta_t & \cos \theta_t & y_t \\ 0 & 0 & 1 \end{bmatrix}. \quad (1)$$

Given a sequence of encoder velocities v_t , IMU yaw rates ω_t , LiDAR scans $\mathcal{Z}_t^L$, and RGBD frames $(\mathcal{D}_t, \mathcal{I}_t)$, we seek to:

- 1) **Localize**: estimate the trajectory $\{T_0^{wb}, T_1^{wb}, \dots, T_N^{wb}\}$ that is globally consistent;
- 2) **Map**: construct a 2-D occupancy grid map $\mathcal{M}$ encoding occupied and free space; and
- 3) **Texture**: build a 2-D color map of the floor from projected RGBD points.

The SLAM problem can be formulated as a maximum a posteriori (MAP) estimation over a factor graph. Defining the state as $\mathbf{x} = (x_0, x_1, \dots, x_N)$ where $x_t = (x_t, y_t, \theta_t)^\top$, we minimize:

$$\mathbf{x}^* = \arg \min_{\mathbf{x}} \sum_{(i,j) \in \mathcal{E}} \|e_{ij}(\mathbf{x})\|_{\Omega_{ij}}^2 \quad (2)$$

where $e_{ij} = z_{ij} \ominus (T_i^{-1}T_j)$ is the error between the measured relative pose z_{ij} and the predicted relative pose, and Ω_{ij} is the information matrix. The edge set $\mathcal{E}$ contains both consecutive odometry/ICP edges and non-consecutive loop closure edges.

III. TECHNICAL APPROACH

A. Encoder and IMU Odometry

The linear velocity v_t is derived from encoder tick counts. Given counts $[FR, FL, RR, RL]$ at 40 Hz, each tick corresponds to 0.0022 m. The right and left wheel distances are $d_R = (FR + RR)/2 \times 0.0022$ and $d_L = (FL + RL)/2 \times 0.0022$, yielding:

$$v_t = \frac{d_R + d_L}{2 \Delta t}. \quad (3)$$

The yaw rate ω_t is taken directly from the IMU gyroscope (z-axis). Using a differential-drive motion model with constant (v, ω) over interval Δt , the relative pose increment in the body frame is:

$$\Delta x = \frac{v}{\omega} \sin(\omega \Delta t), \quad \Delta y = \frac{v}{\omega} (1 - \cos(\omega \Delta t)), \quad \Delta \theta = \omega \Delta t \quad (4)$$

with the degenerate case $\omega \approx 0$ handled as pure translation ($\Delta x = v \Delta t, \Delta y = 0$). The global pose is updated via:

$$T_{t+1}^{wb} = T_t^{wb} \cdot \Delta T_t. \quad (5)$$

B. Point-to-Line ICP Scan Matching

We refine the odometry prediction using point-to-line ICP, which aligns the current LiDAR scan against a local submap. The point-to-line residual for correspondence (p_i, q_i, n_i) is:

$$r_i = n_i^\top (R p_i + t - q_i) \quad (6)$$

where p_i is a source point (body frame), q_i is the closest target point (world frame), n_i is the target surface normal, and (R, t) are the rotation and translation from body to world.

1) *Normal Estimation*: Target normals are estimated via local PCA: for each target point, we find its $k = 10$ nearest neighbors using a KD-tree, compute the 2×2 covariance of the neighborhood, and take the eigenvector corresponding to the smallest eigenvalue as the surface normal. Normals are consistently oriented toward the $+X$ direction.

2) *Gauss-Newton Optimization*: Parameterizing the update as $\xi = (\delta x, \delta y, \delta \theta)^\top$, the Jacobian of r_i with respect to ξ is:

$$J_i = n_i^\top R \begin{bmatrix} 1 & 0 & -p_{iy} \\ 0 & 1 & p_{ix} \end{bmatrix} \quad (7)$$

and we solve the normal equations $H\xi = -g$ where $H = J^\top J$ and $g = J^\top r$. The pose is updated as $T \leftarrow T \cdot \exp(\xi)$ and the process iterates until convergence ($\|\xi\| < 10^{-4}$) or a maximum of 50 iterations.

3) *Degeneracy Check*: Before solving, we verify the Hessian H is well-conditioned by checking its condition number ($< 10^6$) and minimum eigenvalue ($> 10^{-6}$). If the system is degenerate (e.g., in a long corridor with few features), the iteration is rejected.

4) *Sliding-Window Submap*: Rather than scan-to-scan matching, we maintain a sliding window of the last $W = 30$ scans transformed to world frame. Before each ICP iteration, the submap is voxel-downsampled (0.05 m resolution) using vectorized binning, and normals are pre-computed. This provides a richer, more stable target and reduces drift compared to consecutive frame matching.

C. Occupancy Grid Mapping

We construct a 2-D log-odds occupancy grid with 0.05 m resolution. For each LiDAR scan, we transform scan endpoints to the world frame and use Bresenham ray tracing from the sensor origin to each endpoint:

$$L(c) \leftarrow L(c) + \begin{cases} +\lambda_{\text{occ}} & \text{if } c \text{ is the endpoint cell} \\ -\lambda_{\text{free}} & \text{if } c \text{ is along the ray} \end{cases} \quad (8)$$

where $L(c)$ is the log-odds at cell c , $\lambda_{\text{occ}} = 4.0$, and $\lambda_{\text{free}} = 2.0$. Log-odds are clamped to $[-50, 50]$. The occupancy probability is recovered as $P(\text{occ} | c) = 1/(1 + e^{-L(c)})$, and cells with $P > 0.6$ are classified as occupied.

D. Texture Mapping

We build a 2-D color map of the floor from Kinect RGBD data. For each disparity image, the depth is computed as:

$$d_d = -0.00304 d + 3.31, \quad \text{depth} = \frac{1.03}{d_d} \quad (9)$$

where d is the raw disparity value. Each pixel is back-projected to 3-D using the Kinect intrinsics ($f_x = f_y = 585.05$, $c_x = 242.94$, $c_y = 315.84$) and then transformed to the world frame via $T_w^k = T_t^{wb} \cdot T^{bk}$ where T^{bk} encodes the Kinect extrinsic pose (position (0.18, 0.005, 0.36) m, pitch 0.36 rad, yaw 0.021 rad).

Floor points are identified by height thresholding ($|z_w| < 0.15$ m) and projected onto the 2-D grid. RGB values from the corresponding color image are averaged per cell to produce the final texture map.

E. Pose Graph Optimization with Loop Closure

1) *Factor Graph Construction*: The pose graph contains one node per robot pose and two types of edges:

- **Odometry edges** ($i, i+1$): relative pose $z_{ij} = (T_i^{wb})^{-1}T_{i+1}^{wb}$ from ICP, with information matrix $\Omega_{\text{odom}} = \text{diag}(1, 1, 10)$.
- **Loop closure edges** (i, j): verified relative poses between non-consecutive frames revisiting the same location, with strong information $\Omega_{\text{loop}} = \text{diag}(10, 10, 50)$.

2) *Loop Closure Detection*: Candidates are found via proximity-based search: a KD-tree is built from all (x, y) positions, and for each query node i (sampled every k -th node), we find nodes j within radius r meters satisfying $j - i \geq 30$ (to avoid trivial temporal neighbors). Up to 5 nearest candidates per query are retained, and the total set is prioritized by descending time gap to ensure the most informative (temporally distant) pairs survive any candidate cap.

3) *Loop Verification*: Each candidate (i, j) is verified by running point-to-line ICP between scan j and scan i , initialized with $T_{\text{init}} = (T_i^{wb})^{-1}T_j^{wb}$. The loop is accepted only if ICP converges and the fitness (mean squared residual) is below a gate threshold (0.01 for full precision, 0.05 for fast mode with voxel-downsampled scans).

4) *Robust Optimization*: Loop closure edges use Huber robust weighting ($\delta = 1.0$) to limit the influence of incorrect loop closures:

$$w(r) = \begin{cases} 1 & \text{if } \|r\| \leq \delta \\ \delta/\|r\| & \text{otherwise} \end{cases} \quad (10)$$

The pose graph is optimized using Gauss-Newton iterations. The first node is anchored (prior factor with 10^9 diagonal information) to fix the gauge freedom. We also provide a GTSAM backend using `LevenbergMarquardtOptimizer` with `BetweenFactorPose2` and `noiseModel::Robust` for when ABI compatibility allows.

IV. RESULTS

We evaluate our SLAM pipeline on two indoor datasets: Dataset 20 (a closed-loop corridor trajectory with ~ 4960 frames) and Dataset 21 (an open-path exploration through hallways with ~ 4785 frames).

A. Odometry vs. ICP Trajectory

Fig. 1 compares the odometry-only and ICP-refined trajectories on both datasets. The encoder+IMU odometry accumulates significant drift, causing the trajectory to spread laterally by several meters. The scan-to-submap ICP trajectory is considerably tighter: the sliding-window submap provides a stable reference that constrains drift, especially in feature-rich indoor environments.

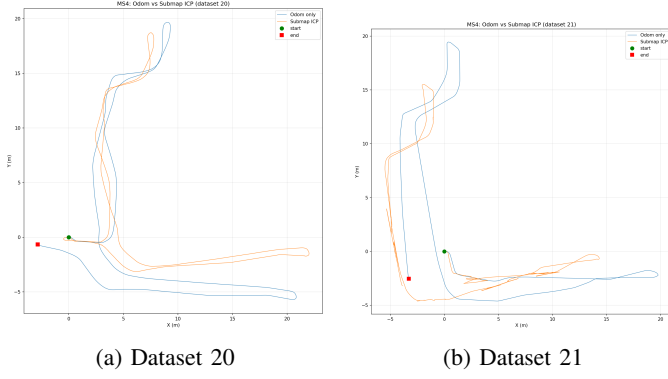


Fig. 1: Odometry-only (blue) vs. scan-to-submap ICP (orange) trajectories. ICP significantly reduces drift and produces more compact, geometrically consistent paths on both datasets.

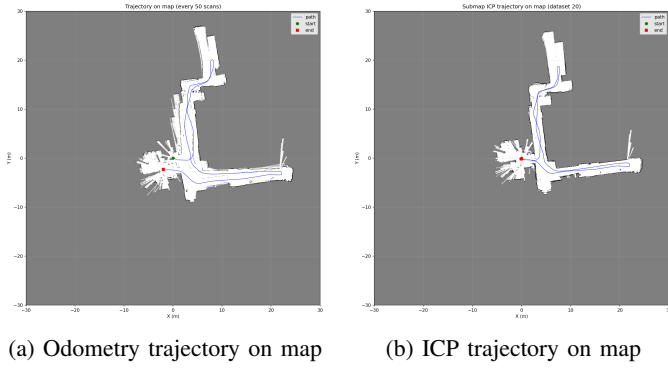


Fig. 2: Trajectory overlaid on occupancy grid maps for Dataset 20. (a) Odometry-only map shows blurred walls and hallway misalignment due to drift. (b) Scan-to-submap ICP map: walls are sharp and corridors have consistent width.

B. Occupancy Grid Mapping

Fig. 2 compares the occupancy grid maps built from odometry-only and ICP-refined trajectories on Dataset 20. The odometry map exhibits blurred wall boundaries and doubled corridors caused by rotational drift. In contrast, the ICP map produces clean wall boundaries and consistent hallway geometry.

C. Loop Closure and Pose Graph Optimization

Fig. 5 illustrates the effect of loop closure and PGO. On Dataset 20, which features a closed-loop path, PGO reduces the start-to-end gap. On Dataset 21, the robot does *not* return to its starting position; PGO still improves consistency where the trajectory re-traverses the same corridors. The proximity-based candidate search (radius 8 m, prioritized by descending time gap) found 300 candidates per dataset, of which 7 and 184 respectively passed ICP verification.

D. Texture Mapping

Fig. 6 shows the 2-D texture maps built by projecting 1000 Kinect RGBD frames onto the occupancy grid using the PGO-optimized trajectory. Individual disparity images are loaded

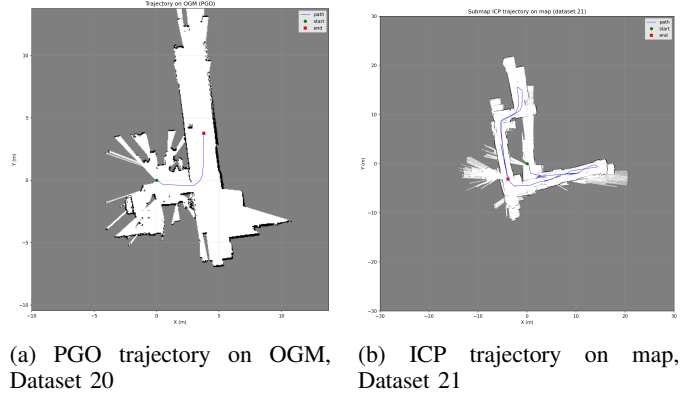


Fig. 3: Trajectories overlaid on occupancy grid maps (first 1000 frames). The path stays within mapped corridors and rooms, confirming accurate localization.

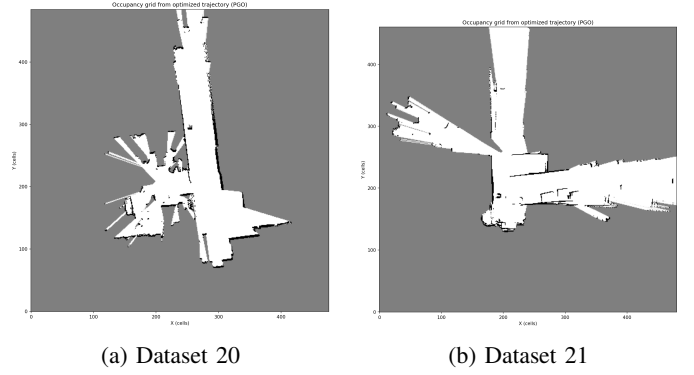


Fig. 4: Final occupancy grid maps from the PGO-optimized trajectory (first 1000 frames). Clear room structure and hallway geometry are visible.

from disk, converted to depth, and back-projected to the world frame using the camera extrinsics. Floor points are identified via height thresholding ($|z_w| < 0.15$ m), and the disparity-to-RGB coordinate mapping corrects for the spatial offset between the depth and color sensors. The resulting maps reveal beige/tan floor tiles, darker corridor sections, and distinct room textures that align well with the occupancy grid geometry. A key implementation detail was composing the camera RPY orientation with a nominal rotation $R_{\text{opt} \rightarrow \text{body}}$ that accounts for the axis swap between the optical camera frame (X-right, Y-down, Z-forward) and the robot body frame (X-forward, Y-left, Z-up).

E. Discussion

What worked well. The scan-to-submap ICP approach substantially reduced drift compared to both odometry-only and scan-to-scan ICP. Point-to-line ICP with KD-tree-based normal estimation provided fast convergence (typically 5–15 iterations per frame). The degeneracy check prevented catastrophic failures in featureless corridors. The sliding window submap ($W=30$) provided a good balance between local consistency and computational cost. Proximity-based loop closure with

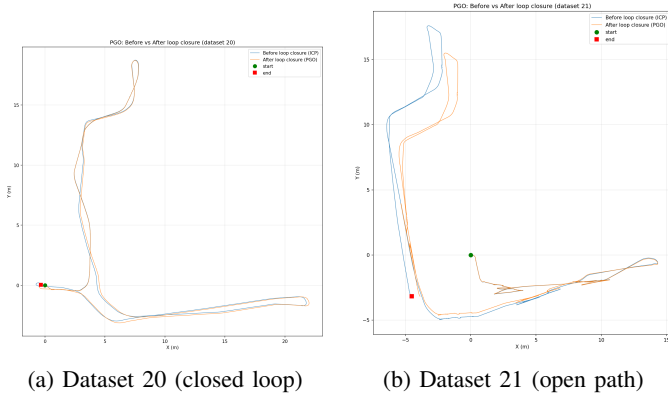


Fig. 5: Before (ICP, blue) vs. after (PGO, orange) loop closure. On Dataset 20, PGO pulls the trajectory end closer to the start. On Dataset 21, PGO improves overlap at re-visited corridor segments.

TABLE I: Loop Closure and PGO Statistics

	Dataset 20	Dataset 21
Total frames	4,962	4,785
Loop candidates	300	300
Verified loops	7	184
PGO edges (odom + loop)	4,961 + 7	4,784 + 184
Initial χ^2	6.26	359.4
Final χ^2 (50 iters)	0.006	2.93
ICP end position	(-0.3, 0.1) m	(-4.5, -3.2) m
PGO end position	(-0.1, 0.0) m	(-3.9, -3.2) m

time-gap prioritization successfully identified meaningful loop constraints.

What was challenging. The initial candidate search radius of 3 m was insufficient for Dataset 21 where ICP drift exceeded 5 m; increasing the radius to 8 m was necessary. Early implementations suffered from a critical bug where ICP correspondences were found in the body frame rather than the transformed frame, effectively preventing proper scan alignment. For texture mapping, the Kinect extrinsic transform required careful handling of the optical-to-body axis convention to correctly project floor points to $z_w \approx 0$.

Performance. Key optimizations included vectorized voxel downsampling ($O(N)$ via `np.unique` and `np.add.at`), KD-tree-based normal estimation ($O(N \log N)$ vs. brute-force $O(N^2)$), and lazy frame-by-frame RGBD loading (avoiding ~ 3.6 GB of memory for batch loading). The full pipeline processes ~ 5000 frames in approximately 18 minutes on a laptop CPU.

V. CONCLUSION

We implemented a complete 2-D LiDAR SLAM system combining differential-drive odometry, point-to-line ICP scan matching with sliding-window submaps, log-odds occupancy grid mapping, RGBD texture mapping, and pose graph optimization with proximity-based loop closure. The scan-to-submap ICP significantly reduces odometry drift, and PGO with loop closure further improves trajectory consistency at

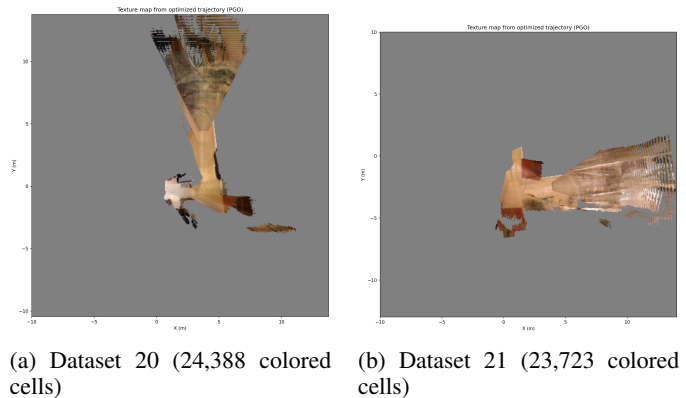


Fig. 6: Floor texture maps from the PGO-optimized trajectory, built by projecting Kinect RGBD data onto the 2-D grid. Beige/tan floor tiles, darker corridor sections, and distinct room boundaries are visible. Grey regions indicate areas without RGBD coverage.

re-visited locations. The resulting occupancy grid maps show clean wall structures and geometrically consistent environments.